

Table des matières

Cours 05 — Le code d'un pipeline robuste	1
Objectifs	2
1. Ce qui sépare un script d'un pipeline	2
2. La configuration hors du code	3
3. Les secrets hors du dépôt	4
4. Une journalisation qui se lit, et qui se compte	4
5. Réessayer ce qui est passager	6
6. Tester une transformation	6
7. Une seule porte d'entrée	7
8. Empaqueter le pipeline	8
Points clés à retenir	9
Exercices	10

Cours 05 — Le code d'un pipeline robuste

Théorie · Module 5/12 — Formation Data Engineering · ~40 min · intermédiaire

Où vous êtes : source → ingestion → lac → entrepôt → restitution, et cette fois **la chaîne elle-même**. Le Cours 05 ne va chercher aucune donnée nouvelle : il reprend ce que les Cours 02 à 04 ont écrit et en fait un programme qu'on lance, relance, surveille et confie à un ordonnanceur, c'est-à-dire au service qui décide quand chaque traitement part et qui prévient quand il échoue. Le Cours 06 en installe un, Airflow.

En 30 secondes :

- Un script devient un pipeline quand il survit à ce qui n'est pas prévu : une valeur qui change de machine, un réseau qui tombe, une source qui répond mal, et quelqu'un qui doit comprendre après coup ce qui s'est passé.
- La configuration sort du code, les secrets sortent du dépôt. Ce sont deux gestes différents, pour deux raisons différentes.
- Un journal se lit. Une trace d'exécution s'interroge. Un pipeline sérieux tient les deux.
- Une erreur passagère se réessaie avec une attente qui double ; une erreur définitive se signale tout de suite. Confondre les deux coûte du temps dans un cas, des données dans l'autre.

- Un test vérifie une promesse sur des données qu’il fabrique, pas sur les 138 Mo de la veille.
- Une seule porte d’entrée en ligne de commande : c’est elle que le conteneur lance, que le DAG appelle, et que le kit de départ d’Analytics Engineering rejoue. Un kit de départ est la commande qui reconstruit, pour la formation suivante, l’état que celle-ci aurait laissé si on l’avait suivie.

L’idée du cours. Les deux cours précédents ont laissé des scripts qui marchent : le bronze en SQL, le chargeur de la RAW en Python, l’extraction ODBC, l’API météo. Ils marchent sur votre poste, lancés à la main, dans l’ordre que vous connaissez. Ici, on en fait un paquet Python avec une seule commande d’entrée, une configuration, des journaux, des reprises, des tests et une image Docker. Rien de nouveau n’est ingéré. Ce qui change, c’est qu’un autre que vous, ou une machine, pourra le lancer.

Objectifs

- Nommer ce qui sépare un script d’un pipeline, et reconnaître les quatre manques les plus coûteux.
- Sortir la configuration du code, et distinguer une valeur de configuration d’un secret.
- Journaliser pour deux lecteurs : celui qui cherche pourquoi, et celui qui compte combien.
- Distinguer une erreur passagère d’une erreur définitive, et borner les reprises.
- Écrire un test qui vérifie une promesse du pipeline, sans les données de production.
- Concevoir une porte d’entrée unique en ligne de commande, et dire pourquoi son code de sortie compte.
- Expliquer ce qu’une image de conteneur fige, et ce qu’elle doit laisser dehors.

1. Ce qui sépare un script d’un pipeline

Labo : Labo_05.md, étape 1.

Le chargeur du Cours 03 fait son travail. Il le fait même bien : il est idempotent, il tient un journal des chargements, il rend des comptes justes. Et pourtant il n’est pas un pipeline, pour quatre raisons qui n’ont rien à voir avec ce qu’il calcule.

Ce qui manque	Ce que ça donne le jour où	Ce que le cours y répond
Des valeurs écrites dans le code	le port de la base change, et il faut modifier un fichier .py pour une valeur qui n’est pas du code	la configuration, section 2

Ce qui manque	Ce que ça donne le jour où	Ce que le cours y répond
Un mot de passe quelque part	le dépôt part chez un collègue, ou sur GitHub, avec le mot de passe dedans	les secrets, section 3
Des print	le pipeline a tourné cette nuit, personne ne regardait, et il ne reste rien	la journalisation, section 4
Un appel réseau sans filet	l'API répond 503 pendant huit secondes, et toute l'exécution est perdue	les reprises, section 5

Un cinquième manque se voit moins, parce qu'il ne fait rien planter : rien ne prouve que le code fait ce qu'il dit. On le corrige par des tests (section 6). Et un sixième apparaît dès qu'une machine doit lancer le travail à votre place : il faut une commande, une seule, qui rende un code de sortie (section 7) et qui tourne ailleurs que chez vous (section 8).

À retenir — robuste ne veut pas dire compliqué. Chacune des six réponses de ce cours tient en quelques dizaines de lignes. Ce qui coûte cher, c'est de les ajouter après, sur un code qui a grossi sans elles.

2. La configuration hors du code

Labo : Labo_05.md, étape 2.

Une valeur de configuration est une valeur qui change d'un environnement à l'autre sans que le programme change : un port, un chemin, une station de météo, un nombre d'essais. Une valeur du domaine, elle, fait partie du sens du programme : les 22 livraisons de Boréal et la forme de leurs fichiers ne sont pas de la configuration, ce sont les données que le pipeline connaît.

La distinction se teste par une question : est-ce que cette valeur serait différente sur le poste d'un collègue, sur un serveur de préproduction, en production ? Si oui, elle sort. Sinon, elle reste, et la sortir ne ferait que disperser la lecture.

Forme	Ce qu'elle porte bien	Ce qu'elle porte mal
Un fichier de configuration (<code>config.toml</code>)	ce qui vaut pour tout le monde, structuré, commenté, versionné	ce qui change à chaque lancement, et tout secret
Une variable d'environnement	ce qui change d'une machine ou d'un conteneur à l'autre	une structure, une liste longue, une valeur qu'on veut relire dans six mois
Un argument de ligne de commande	ce qui change à chaque lancement : la date à traiter, l'étape à lancer	ce qui ne change jamais, et qu'on retaperait chaque fois

Les trois se superposent, du plus général au plus précis : le fichier donne la valeur par défaut, l'environnement la remplace sur cette machine-là, l'argument la remplace pour ce lancement-là. Le pipeline de Boréal lit `config.toml` par le module `tomllib`, entré dans la bibliothèque standard de Python avec la version 3.11, puis laisse une variable `PIPELINE_ERP_PORT` gagner sur `[erp] port`.

Le format TOML est retenu pour une raison précise : il ne demande rien à installer. Le chemin que rejoue le kit de départ d'Analytics Engineering n'importe que DuckDB et la bibliothèque standard, et une dépendance de plus serait une dépendance de plus à installer chez celui qui reçoit la livraison. YAML, plus répandu, reviendra au Cours 11 pour les contrats de données, où sa souplesse sert.

3. Les secrets hors du dépôt

Labo : Labo_05.md, étapes 2 et 8.

Un secret n'est pas une configuration particulièrement sensible : c'est autre chose. Une configuration se lit, se compare, se versionne, et son historique est utile. Un secret ne doit avoir aucun historique, parce qu'un dépôt Git n'oublie rien : un mot de passe retiré dans le commit suivant reste lisible dans le commit précédent, et le retirer vraiment demande de réécrire l'historique.

Trois règles suffisent pour Boréal, et elles suffisent aussi en entreprise, à l'échelle près.

Règle	Chez Boréal	À plus grande échelle
Le secret vit hors du dépôt	<code>01_de/.env</code> , nommé dans le <code>.gitignore</code> du Cours 01	un gestionnaire de secrets, interrogé au démarrage
Le dépôt garde un modèle, jamais une valeur	<code>.env.exemple</code> , avec <code>ERP_MOT_DE_PASSE=remplacez-moi</code>	le même modèle, plus la procédure d'obtention
Le code ne connaît que le nom du secret	<code>secret("ERP_MOT_DE_PASSE")</code>	identique : seule la source de la valeur change

La troisième règle est celle qui rend les deux autres tenables. Tant que le code demande une valeur par son nom, l'endroit où cette valeur dort peut changer sans que le code bouge : un `.env` sur votre poste, une variable d'environnement dans le conteneur, un gestionnaire de secrets en production.

Piège — le secret qui entre par la porte de derrière. Un mot de passe passé en argument de ligne de commande se retrouve dans l'historique du terminal et dans la liste des processus de la machine, où n'importe quel compte peut le lire. Un secret entre par un fichier ou par l'environnement, jamais par la ligne de commande.

4. Une journalisation qui se lit, et qui se compte

Labo : Labo_05.md, étape 3.

`print` écrit à l'écran. C'est suffisant tant que quelqu'un regarde l'écran. Dès que le pipeline tourne la nuit, dans un conteneur, appelé par un ordonnanceur, il faut autre chose, et il faut répondre à deux questions de nature différente.

Question	Qui la pose	Ce qu'il lui faut
Pourquoi cette exécution a-t-elle échoué ?	celui qui répare, une fois	un texte daté, avec le niveau de gravité et l'étape qui parle
Combien de fois, depuis quand, avec quel écart ?	celui qui surveille, tous les jours	des lignes de table, qu'on interroge en SQL

Le module `logging` de Python répond à la première. Il ajoute à chaque ligne son heure, son niveau et son origine, il écrit au même moment à la console et dans un fichier, et il laisse régler à partir de quel niveau une ligne s'écrit. Les niveaux se choisissent comme ceci, et le choix n'est pas décoratif : un journal où tout est en `ERROR` ne se lit pas plus qu'un journal où rien ne l'est.

Niveau	Ce qu'on y met chez Boréal
DEBUG	le détail qui n'intéresse que pendant qu'on cherche : chaque page d'API, chaque table
INFO	le déroulement normal : l'étape commence, l'étape finit, avec ses comptes
WARNING	ce qui a été rattrapé : un essai raté, une reprise
ERROR	ce qui a arrêté le travail

La table `pipeline.executions` répond à la seconde. Chaque passage d'une étape y inscrit une ligne : son identifiant, l'étape, le début, la fin, la durée, l'état, les lignes écrites, et le message de l'erreur s'il y en a une. Elle vit dans le schéma `pipeline` de l'entrepôt, à côté du journal des chargements du Cours 03, et se lit comme n'importe quelle table.

La ligne s'écrit **avant** que l'étape commence, à l'état « en cours », puis se complète à la fin. Ce détail a une raison : si le processus est tué en plein travail, la ligne reste à « en cours », et une exécution qui n'a jamais fini se voit. Une ligne écrite seulement à la fin disparaîtrait avec le processus, et la table dirait que rien ne s'est passé cette nuit-là.

À retenir — les deux traces ne se remplacent pas. Le fichier de journal dit ce qui s'est passé, dans l'ordre, avec les détails. La table dit ce qui se passe d'habitude, et permet de comparer. Le Cours 11 partira de cette table pour l'observabilité, et le Cours 06 s'en servira pour savoir ce qu'un DAG a vraiment fait.

Une trace qui vit dans la base qu'elle surveille pose un problème que le fichier de journal n'a pas. L'étape travaille dans une transaction ; si elle échoue en plein milieu, cette transaction est avortée, et tout ce qui passerait par la même connexion échoue avec elle, y compris la ligne qui dirait pourquoi. Le résultat serait le pire des deux mondes : la vraie cause perdue, et une ligne restée à « en cours » qui prétend que l'étape tourne encore. Le remède tient en deux règles. Qui ouvre une transaction la referme, même quand ça rate : un `ROLLBACK` dans le chemin d'erreur, là où le `BEGIN` a été écrit. Et la trace se ferme hors de toute transaction, sous son propre garde-fou : si elle n'arrive pas à s'écrire, c'est la cause réelle qui doit remonter, jamais l'erreur de journalisation.

Pourquoi — pourquoi les outils d'observabilité vivent ailleurs. C'est la même raison, en plus grand. Un système qui garde son état de santé dans la base qu'il surveille perd les deux d'un coup le jour où la base tombe. Ici, le compromis est assumé : la table sert à comparer des exécutions, pas à sonner l'alarme, et le fichier de journal, lui, est ailleurs. Le Cours 11 y revient.

5. Réessayer ce qui est passager

Labo : Labo_05.md, étape 4.

L'extraction météo du Cours 04 réessayait déjà : c'était la seule étape qui touchait le réseau. Le Cours 05 sort cette boucle du module de la météo et en fait un outil commun, parce que la question se reposera au Cours 09 pour le courtier MQTT et au Cours 06 pour toute tâche d'un DAG.

Tout tient dans une distinction.

Nature de l'erreur	Exemples	Ce qu'on fait
Passagère	coupure du réseau, délai dépassé, 429 (trop de requêtes), 500 à 504 du serveur	réessayer, quelques fois, en attendant de plus en plus
Définitive	401 (pas authentifié), 403 (pas autorisé), 404 (n'existe pas), une requête mal formée	abandonner tout de suite et le dire

Réessayer une erreur définitive ajoute du délai et du bruit, sans jamais rien changer. Abandonner sur une erreur passagère jette une exécution entière pour trois secondes de réseau.

L'attente double à chaque échec : 2 secondes, puis 4, puis 8. Cette temporisation exponentielle existe autant pour la source que pour nous. Un service déjà en difficulté n'a pas besoin de dix clients qui le rappellent chaque seconde ; en lui laissant de l'air, on augmente la chance qu'il se rétablisse, donc la nôtre d'obtenir une réponse.

Le nombre d'essais est borné, et c'est une décision, pas un oubli : quatre essais, trois attentes, quatorze secondes en tout. Passé ce point, le pipeline échoue et le dit. Une boucle infinie de reprises ne rend pas un pipeline robuste, elle transforme une panne visible en panne silencieuse qui tient la place jusqu'au matin.

Piège — réessayer ce qui n'est pas rejouable. Une reprise suppose que refaire l'appel est sans conséquence. C'est vrai d'une lecture ; ce n'est pas vrai d'une écriture qui ajoute. C'est pour cela que l'idempotence du Cours 03 vient avant les reprises de celui-ci : le chargeur peut se relancer parce que son journal l'empêche de doubler quoi que ce soit.

6. Tester une transformation

Labo : Labo_05.md, étape 6.

Un pipeline se trompe rarement en plantant. Il se trompe en rendant un chiffre faux, sans la moindre erreur. Les requêtes de contrôle des Cours 02 à 04 attrapent cela sur les données réelles ; les tests l'attrapent avant, sur des données minuscules, en une fraction de seconde.

Trois tests suffisent à couvrir les promesses du pipeline de Boréal, et chacun vérifie une propriété, pas une ligne de code.

Le test	La promesse qu'il tient	Ce qu'il fabrique
Le journal refuse un extrait déjà chargé	relancer ne double rien ; un fichier corrigé remplace ses lignes ; un autre nom est une autre livraison	trois petits CSV de deux ou trois lignes
La conversion garde le compte des lignes	ce qui entre en CSV ressort en Parquet, sans perte ni doublon	deux CSV et un script de conversion réduit
La reprise s'arrête après le nombre d'essais prévu	quatre essais, trois attentes qui doublent, puis l'abandon ; un 404 ne passe qu'une fois	une fonction qui échoue toujours, et une horloge factice

Deux principes rendent ces tests utiles plutôt que décoratifs.

Le premier : le test fabrique ses données. Il ne dépend ni de la génération de Boréal, ni d'un conteneur, ni du réseau, ce qui lui permet de tourner à chaque changement, en moins d'une seconde. Un test qui demande deux minutes de mise en place ne sera pas lancé.

Le second : le test n'écrit rien dans le projet. Le pipeline reçoit, par une variable d'environnement, un dossier temporaire que l'outil de test efface ensuite. C'est d'ailleurs la même variable qui sert au conteneur, section 8 : ce qui rend un programme testable le rend souvent transportable, et l'inverse est vrai aussi.

Ce qui ne se teste pas ainsi, on le nomme : la vraie API d'ECCC, le vrai pilote ODBC, les 138 Mo de Boréal. Ces chemins se vérifient en les exécutant, avec les contrôles des labos, et non par un test automatique qui dépendrait d'un service extérieur.

7. Une seule porte d'entrée

Labo : Labo_05.md, étape 5.

C'est la règle de conception qui sert tout le reste du parcours : le pipeline s'appelle d'une seule façon.

```
python -m pipeline bronze raw corpus
```

Trois consommateurs différents passent par cette même porte, et aucun n'a besoin de savoir ce qu'il y a derrière.

Qui appelle	Comment	Quand
Vous, dans le terminal	<code>python -m pipeline raw</code>	pendant que vous travaillez
Le conteneur	la commande est le point d'entrée de l'image	section 8
Le DAG d'Airflow	une tâche par étape, qui lance la même commande	Cours 06

Qui appelle	Comment	Quand
Le kit de départ d'Analytics Engineering	la même commande, sans Airflow	Cours 12

Deux détails de cette porte comptent autant que le reste.

Le code de sortie, d'abord : 0 quand tout est passé, 1 quand une étape a échoué. C'est la seule chose qu'un ordonnanceur lit avec certitude. Un pipeline qui écrit « ERREUR » dans son journal et rend 0 sera compté comme réussi, et personne ne sera prévenu.

L'ordre, ensuite. Les étapes qui demandent le réseau ou un conteneur ne font pas partie de ce que lance tout : un pipeline qu'on doit pouvoir rejouer hors ligne ne suppose ni l'API d'ECCC ni la base de l'ERP. Elles restent appelables par leur nom, quand leurs conditions sont réunies.

8. Empaqueter le pipeline

Labo : Labo_05.md, étape 7.

Deux mots d'abord, parce que tout ce qui suit s'appuie dessus. Une image est un paquet figé qui contient un logiciel et tout ce dont il a besoin pour tourner. Un conteneur est une instance de cette image, en marche sur votre machine. On ne modifie pas un conteneur à la main : on le détruit et on le recrée depuis l'image, ce qui garantit que tout le monde fait tourner exactement la même chose. C'est aussi pour cette raison que la base de l'ERP du Cours 04 se recharge par son script plutôt qu'à la main.

Analogie — l'image et le conteneur. L'image est le plan de l'ouvrage ; le conteneur est l'ouvrage bâti d'après ce plan. On en bâtit autant qu'on veut, on les démolit sans toucher au plan, et deux ouvrages bâtis d'après le même plan sont identiques.

Une image de conteneur est donc une réponse à « ça marche sur ma machine ». Elle fige la version de Python, celle de DuckDB et le code, si bien que le même pipeline rend les mêmes chiffres chez vous, chez un collègue et sur le serveur qui l'ordonnancera au Cours 06.

Ce qu'elle fige et ce qu'elle laisse dehors se décide ligne par ligne.

Dans l'image	Dehors, et pourquoi
Python, dans une version nommée	les données : elles arrivent par un montage, et une image de 138 Mo de données ne se reconstruit pas
DuckDB, dans une version fixée (duckdb==1.5.5)	les secrets : ils entrent par l'environnement, au lancement
Le paquet pipeline et les scripts SQL	les tests : ils appartiennent à l'atelier, pas au pipeline livré
La configuration par défaut	les journaux : ils s'écrivent dans le dossier monté, et survivent au conteneur

Fixer la version de DuckDB mérite un mot. Sans elle, l'image reconstruite dans six mois prendra la version du jour, et un moteur SQL qui change de version peut changer un type deviné ou un ordre de tri. Le Cours 01 a montré ce que la détection de types fait aux extraits de l'ERP : ce n'est pas le genre de chose qu'on veut voir bouger sans l'avoir décidé.

Le pipeline trouve le projet par une variable d'environnement, `PIPELINE_PROJET`, qui vaut `/travail` dans l'image. C'est la même variable dont les tests se servent pour travailler dans un dossier temporaire. Un programme qui ne suppose pas où il tourne se déplace sans se réécrire.

Piège — le conteneur ne vit pas à la même heure que vous. Un conteneur Linux tourne en temps universel coordonné, UTC, tant qu'on ne lui dit pas autre chose. Le fichier de journal qu'il écrit porte donc parfois la date du lendemain : le labo en fait la démonstration, et c'est une bonne raison de dater les journaux en UTC partout plutôt que de deviner.

Recoupements — ce que ce cours partage avec ses voisines. - Python. Le langage lui-même, ses paquets, ses environnements virtuels et son outillage appartiennent à la formation Python. Ici, seulement ce qui rend un pipeline rejouable : la configuration, les journaux, les reprises, les tests, la porte d'entrée. - Linux. Le terminal, les variables d'environnement et les droits sur les fichiers sont la matière de la formation Linux. Ici, on s'en sert sans les enseigner.

Points clés à retenir

- Un script devient un pipeline quand il survit à ce qui n'était pas prévu : une valeur qui change, un réseau qui tombe, une nuit sans témoin.
 - La configuration sort du code parce qu'elle change d'un environnement à l'autre. Le secret sort du dépôt parce que Git n'oublie rien. Ce sont deux gestes distincts.
 - Le code ne connaît que le nom d'un secret : ce qui change d'un environnement à l'autre, c'est l'endroit où la valeur dort.
 - Un journal de texte répond à « pourquoi », une table d'exécutions répond à « combien de fois ». Un pipeline sérieux tient les deux, et la ligne s'écrit avant le travail, pas après.
 - Une erreur passagère se réessaie avec une attente qui double, un nombre borné de fois. Une erreur définitive s'annonce tout de suite. Et l'on ne réessaie que ce qui est rejouable.
 - Un test vérifie une promesse sur des données qu'il fabrique, en moins d'une seconde, sans toucher au projet. Ce qui dépend d'un service extérieur se vérifie autrement, et on le dit.
 - Une seule porte d'entrée en ligne de commande, avec un code de sortie honnête : c'est ce que le conteneur lance, ce que le DAG appelle, et ce que le kit rejoue.
 - Une image fige les versions et laisse dehors les données, les secrets et les tests.
-

Exercices

Les exercices de ce cours vivent dans `Exercices_05.md`, et leur corrigé dans `Corrige_05.md`. Faites-les après le labo : plusieurs énoncés supposent que le paquet `pipeline` existe, que ses tests passent et que la table `pipeline.executions` a été remplie par quelques passages.